Git
GitHub

Introduction to Git

- Git is an open-source distributed version control system designed to help developers manage and track changes to their code over time. Unlike older centralized version control systems, Git is distributed, allowing each developer to have a full copy of the code base on their local machine. This enables offline work and seamless switching between different versions of the project.

What does Git do?

Manage projects with Repositories

Clone a project to work on a local copy

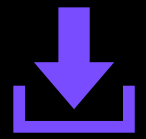
Control and track changes with Staging and Committing

Branch and Merge to allow for work on different parts and versions of a project

Pull the latest version of the project to a local copy

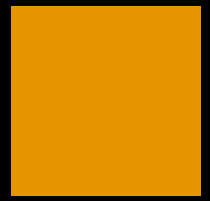
Push local updates to the main project

Installing Git



download Git for free from the following website: https://www.git-scm.com/

To confirm that Git is installed correctly, open your terminal or command prompt and enter the following command:



```
git --version
```

Setting Up Git

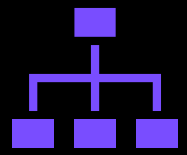
- Once Git is installed, the first step is to set your username and email, which will be attached to your commits. Open your terminal or command prompt and enter the following commands, replacing “Your Name” with your actual name and “your.email@domain.com” with your email address:
- To configure the remote repository, you would typically use the following commands:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@domain.com"
```

Initialize Git

- To start using Git, you first need to initialize a repository, which is a designated folder that Git will track for version control. Navigate to your project directory using the terminal or command prompt and run the following command:
- **`git init`**

Git Staging Environment



One of the core functions of Git is the concepts of the Staging Environment, and the Commit.

Staged files are files that are ready to be committed to the repository you are working on.

Git Adding New Files

- The first command you'll need is git add, which adds files to the staging area to be committed. For example, to add a file named index.html, you would run the following command:

- `git add index.html`

Git Commit

- After adding the file to the staging area, you can make a commit using the git commit command. A commit is a record of the changes you have made to the repository. To commit the staged files with a meaningful message, use the following command:

- **`git commit -m "Add initial homepage"`**

Git Status

- You can also use the git status command to check the status of your repository, including which files are staged, unstaged, or untracked. For example:
- `git status`

Git Commit Log

- To view the history of commits for a repository, you can use the log command:

- **`git log`**

Git Help

- If you are having trouble remembering commands or options for commands, you can use Git help.

- **`git help --all`**

Git Branch

- To create a new branch, use the git branch command followed by the desired branch name. For example, to create a branch named new-feature, run the following command:
- **`git branch new-feature`**

- Once you have created a branch, you can start working on it by switching to the branch using the git checkout command:
- **`git checkout new-feature`**

Git Branch

- When your feature is complete, you can merge the changes back into the main branch. To do this, switch to the main branch using git checkout and then run the following command:
- **`git merge new-feature`**

Working with Remote Repositories on GitHub

- Once you have created the repository, you can connect your local repository to the remote using the following command:
- **`git remote add origin <Repo Url>`**

Push Local Repository to GitHub

- You can then push your code to GitHub using the git push command:
- **`git push -u origin main`**
- This command pushes the main branch to the origin remote repository. The `-u` flag sets the main branch as the default remote branch. To pull the latest code from GitHub, use the following command:
- **`git pull origin main`**
- This command fetches and merges changes from the remote main branch into your local repository.

Push Changes to GitHub

- Let's try making some changes to our local git and pushing them to GitHub.
- To add all staged changes following command
- **`git add .`**
- **`git commit -m "Commit message"`**
- **`git push origin main`**

Git Clone from GitHub

- To clone repo from github following command:

- `git clone <repo url>`

Renaming the branch

- `git branch -m <old-name> <new-name>`
- To Rename the current branch
- `git branch -m new-branch-name`

- After renaming, the new branch does not exist on the remote yet.
- To push the new branch name to the remote:
- **`git push origin new-branch-name`**
- The old branch still exists on the remote, so you need to delete it:
- **`git push origin --delete old-branch-name`**

Git Ignore and .gitignore

- When sharing your code with others, there are often files or parts of your project, you do not want to share.
- To create a .gitignore file, go to the root of your local Git, and create it:
- **`touch .gitignore`**

```
# ignore ALL .log files
*.log

# ignore ALL files in ANY directory named temp
temp/
```

Git stash

- The git stash command temporarily saves (or "stashes") changes in your working directory that you’re not ready to commit yet, allowing you to work on something else. Later, you can reapply the stashed changes when you're ready.

Basic Syntax :

- `git stash`

1. Stash Changes

- This command stashes both modified tracked files and staged changes.
- `git stash`
- This moves your changes (both staged and unstaged) into a stash, and reverts your working directory to the last commit (clean state).
- Use git status after stashing, and you'll see that your working directory is clean.

2. View Stashed Changes

- `git stash list`

3. Apply the Last Stash

- Reapply the most recent stashed changes to your working directory without removing it from the stash.

- `git stash apply`

- This restores the stashed changes, but keeps them in the stash list.

4. Apply and Remove the Stash

- Reapply the stash and remove it from the stash list:

- `git stash pop`

5.Clear All Stashes

- To remove all stashed changes:

- `git stash clear`

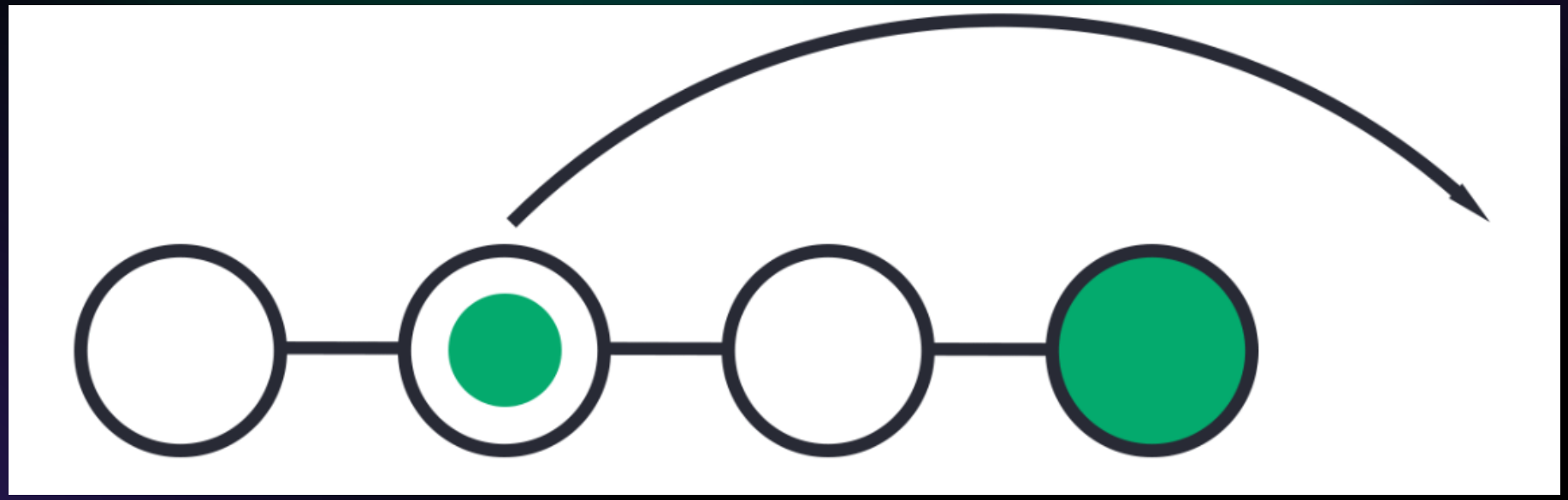
6. Drop a Stash

- Remove a specific stash:

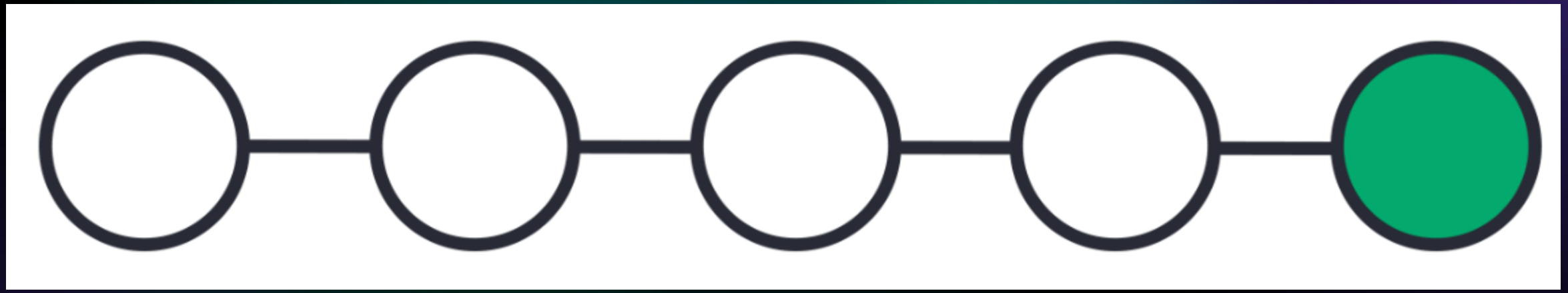
- **git stash drop stash@{0}**

Git Revert

- revert is the command we use when we want to take a previous commit and add it as a new commit, keeping the log intact.
- Step 1: Find the previous commit:



- Step 2: Use it to make a new commit:



Syntax

- **`git revert <commit-hash>`**
- Example :
- View commit history to find the commit hash:
- **`git log`**
- Example output:

```
commit abc123def456
Author: Your Name <you@example.com>
Date:   Mon Sep 21 2023

Commit message for the changes
```

- Revert the commit:
- `git revert abc123def456`
- Push the changes to the remote:
- `git push origin <branch-name>`
- Revert Multiple Commits
- `git revert <commit-hash1> <commit-hash2> ...`

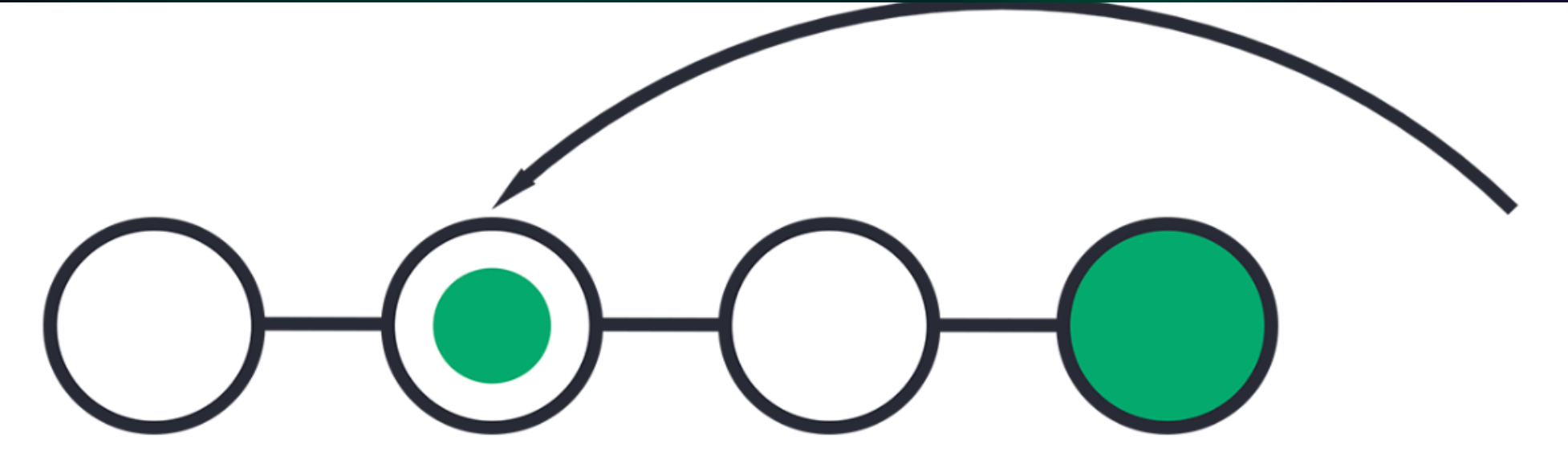
Revert a Merge Commit

- To revert a merge commit, specify the parent branch using -m:
- **`git revert -m 1 <merge-commit-hash>`**

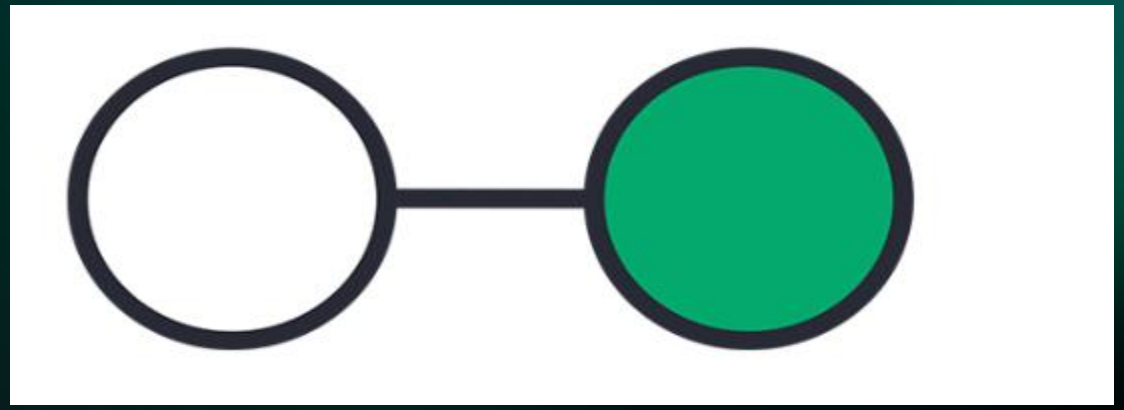
Git Reset

- reset is the command we use when we want to move the repository back to a previous commit, discarding any changes made after that commit.

- Step 1: Find the previous commit:



- Step 2: Move the repository back to that step:



Basic Syntax

- `git reset [mode] <commit-hash>`

Modes:

- --soft: Moves the HEAD to the specified commit but keeps the changes in the staging area.
- --mixed (default): Moves the HEAD and unstages the changes (keeps them in the working directory).
- --hard: Moves the HEAD, unstages and deletes changes from the working directory.

1. Soft Reset

- Purpose: Undo commits but keep changes in the staging area.

Syntax

- `git reset --soft <commit-hash>`

Example:

- Let's say you want to undo the last commit but keep the changes:
- `git reset --soft HEAD~1`
- This moves the HEAD back by one commit, but your changes remain staged.

2.Mixed Reset (Default)

- Purpose: Undo commits and unstage changes (keep them in the working directory).
- **`git reset --mixed <commit-hash>`**

Example:

- Undo the last commit and unstage the changes:
- **`git reset HEAD~1`**
- This will remove the last commit, but the changes will still be in your working directory, ready to be edited or re-staged.

3. Hard Reset

- Purpose: Completely remove commits and changes from both the working directory and staging area
- **`git reset --hard <commit-hash>`**

Example:

- Undo the last commit and discard changes:
- **`git reset --hard HEAD~1`**
- This will delete both the commit and the changes made in that commit. Be careful, as this cannot be undone easily.

Reset to a Specific Commit

- You can reset to any specific commit by using its commit hash.

- **`git reset --hard abc123def456`**

4. Reset a Single File

- You can reset a specific file to its previous state without affecting other files.
- `git reset <file-name>`

- `Example:`
- `git reset file.txt`

Git Best Practices

- Write clear, concise commit messages explaining why the changes were made.
- Keep a linear commit history using rebases instead of merges to maintain a clean and readable history.
- Test your code locally before pushing to avoid breaking the build or introducing bugs.